CHAPTER 06

NoSQL Databases

Redis · MongoDB · Cassandra · Neo4j · Elasticsearch · CAP

Chapter Overview

Chapter 06 covers NoSQL databases used in ShopVerse: Redis (caching, sessions, rate limiting, pub/sub, Lua scripts, streams), MongoDB (product catalog, reviews, aggregation pipeline), Cassandra (time-series order events, wide-column model), Neo4j (product recommendations, graph queries), Elasticsearch (full-text search), the CAP theorem, and the decision framework for choosing the right store.

Sub-Chapter	Database	ShopVerse Use Case
06-01	Redis – Core Data Structures	Session store, product cache, cart, rate limiter
06-02	Redis – Streams & Pub/Sub	Order event stream, real-time notifications, consumer groups
06-03	Redis – Lua Scripts & Patterns	Atomic rate limit, flash-sale countdown, Bloom filter
06-04	MongoDB – Document Model	Product catalog, flexible attributes, reviews, aggregation
06-05	MongoDB – Aggregation Pipeline	$match, $group, $lookup, $facet, $bucket, $unwind
06-06	Cassandra – Wide-Column Model	Order event log, time-series metrics, CQL schema design
06-07	Neo4j – Graph Database	Product recommendations, Cypher queries, relationship properties
06-08	Elasticsearch	Full-text search, faceting, autocomplete, index mapping
06-09	CAP Theorem & Consistency Models	CP vs AP, eventual consistency, read-your-writes
06-10	SQL vs NoSQL Decision Matrix	When to use each; ShopVerse multi-database rationale
06-11	Data Modelling Comparison	E-R vs Document vs Wide-Column vs Graph for same domain
06-12	Redis Cluster & High Availability	Sentinel, Cluster mode, read replicas, persistence

06-01 Redis – Core Data Structures

Structure	Key Commands	TTL?	ShopVerse Usage
String	GET, SET, INCR, SETNX, GETSET	Yes	Session tokens, JWT blacklist, feature flags, counters
Hash	HGET, HSET, HMSET, HGETALL, HDEL	Yes	Cart: cart:{userId} → {product:{id}: qty}
List	RPUSH, LPOP, LRANGE, LTRIM	Yes	Recent orders list, FIFO notification queue
Set	SADD, SMEMBERS, SISMEMBER, SUNION	Yes	Active sessions per user, product tag sets
Sorted Set	ZADD, ZRANGE, ZRANGEBYSCORE, ZRANK	Yes	Leaderboard (score=sales), sliding-window rate limit
Bitmap	SETBIT, GETBIT, BITCOUNT	Yes	Daily active users (1 bit per userId per day)
HyperLogLog	PFADD, PFCOUNT, PFMERGE	Yes	Unique visitor count (≈0.81% error)
Stream	XADD, XREAD, XGROUP, XACK	Yes	Event log with consumer groups (Ch06-02)

```
// 1. Product Cache (String – serialised JSON)
SET product:1234 '{...json...}' EX 3600 // cache for 1 hour
GET product:1234 // O(1) lookup
GETDEL product:1234 // get and delete atomically
// 2. Shopping Cart (Hash – one key per user)
HSET cart:42 product:1001 2 product:1002 1 // set quantities
HINCRBY cart:42 product:1001 1 // increment qty by 1
HDEL cart:42 product:1001 // remove item
HGETALL cart:42 // get full cart
EXPIRE cart:42 86400 // 24h TTL
// 3. Sorted Set Rate Limiter
ZADD rate:user:42 1718000000000 "req-uuid-1" // score=epoch-ms
ZREMRANGEBYSCORE rate:user:42 0 (now-60000) // remove entries >60s old
ZCARD rate:user:42 // count requests in last 60s
```

■ Always set TTL on Redis keys. Without TTL, memory grows unbounded. Use SCAN instead of KEYS for production key enumeration – KEYS blocks the server.

Redis Streams for Order Events

```
// Producer: add event to stream
XADD order-events * event_type ORDER_PLACED order_id 1001 customer_id 42 total 2999
// * = auto-generated ID: {millsecond}-{sequence}
// Consumer group: multiple consumers process stream partitioned
XGROUP CREATE order-events notification-group $ MKSTREAM
XGROUP CREATE order-events inventory-group $ MKSTREAM
// Consumer reads pending messages
XREADGROUP GROUP notification-group consumer-1
COUNT 10 BLOCK 2000 STREAMS order-events >
// > means "new messages not yet delivered to this group"
// Acknowledge processed message
XACK order-events notification-group {message-id}
// Java: Spring Data Redis stream listener
@Bean public StreamMessageListenerContainer>
orderEventStreamContainer(RedisConnectionFactory f) {
var options = StreamMessageListenerContainer.StreamMessageListenerContainerOptions
.builder()
.targetType(OrderEvent.class)
.build();
return StreamMessageListenerContainer.create(f, options);
}
```

Pub/Sub vs Streams Comparison

Feature	Pub/Sub	Streams
Message persistence	No – fire and forget	Yes – stored until XDEL or trimmed
Consumer groups	No – all subscribers get all messages	Yes – each message consumed by one group member
Message replay	No	Yes – XRANGE from any ID
Backpressure	No	Yes – MAXLEN trim
ShopVerse use	Real-time WebSocket push	Order event log with guaranteed delivery

Atomic Rate Limiter (Sliding Window)

```
-- Lua script executes atomically (no interleaving)
local key = KEYS[1]
local limit = tonumber(ARGV[1])
local window = tonumber(ARGV[2]) -- seconds
local now = tonumber(ARGV[3]) -- epoch ms
local cutoff = now - window * 1000
redis.call("ZREMRANGEBYSCORE", key, 0, cutoff)
local count = redis.call("ZCARD", key)
if count < limit then
redis.call("ZADD", key, now, now)
redis.call("EXPIRE", key, window)
return 1 -- request allowed
else
return 0 -- rate limited
end
```

```
// Java: load and execute Lua script
@Bean public RedisScript rateLimitScript() {
return RedisScript.of(RATE_LIMIT_LUA, Long.class);
}
public boolean isAllowed(String userId) {
Long result = redisTemplate.execute(rateLimitScript,
List.of("rate:" + userId),
String.valueOf(MAX_REQUESTS),
String.valueOf(WINDOW_SECONDS),
String.valueOf(System.currentTimeMillis()));
return Long.valueOf(1L).equals(result);
}
```

Flash Sale Inventory (Atomic Decrement)

```
-- Stock reservation: atomic check-and-decrement
local key = KEYS[1] -- e.g. flash:stock:product:1001
local qty = tonumber(ARGV[1])
local stock = tonumber(redis.call("GET", key) or "0")
if stock >= qty then
redis.call("DECRBY", key, qty)
return qty -- reserved successfully
else
return 0 -- insufficient stock
end
```

Pattern	Command(s)	Atomicity	Use Case
Check-and-set	WATCH + MULTI/EXEC	Optimistic	Conditional updates; retry on conflict
Atomic increment	INCR / INCRBY	Yes (single cmd)	Counter, page views, stock decrement
Lua script	EVAL / EVALSHA	Yes (script atomic)	Complex multi-step logic without race
SETNX (mutex)	SET key val NX EX ttl	Yes	Distributed lock acquisition

Product Document Structure

```
{
"_id": "prod_1234",
"name": "Sony WH-1000XM5 Headphones",
"category": "electronics/headphones",
"basePrice": 29999, "currency": "INR",
"attributes": {
"brand": "Sony", "connectivity": "Bluetooth 5.2",
"batteryLife": "30h", "noiseCancellation": true
},
"variants": [
{ "sku": "WH5-BLK", "color": "Black", "stock": 45 },
{ "sku": "WH5-SLV", "color": "Silver", "stock": 12 }
],
"ratings": { "avg": 4.7, "count": 2341 },
"tags": ["wireless","premium","anc"],
"createdAt": ISODate("2025-01-15"),
"updatedAt": ISODate("2025-06-01")
}
```

MongoDB Index Types

Index Type	Command	Use Case
Single field	db.products.createIndex({category:1})	Filter by category
Compound	db.products.createIndex({category:1,price:1})	Filter + sort on multiple fields
Text index	db.products.createIndex({name:"text",description:"text"})	Full-text search
Multikey	db.products.createIndex({"tags":1})	Array field – one entry per element
Geospatial 2dsphere	db.stores.createIndex({location:"2dsphere"})	Find nearby stores
TTL index	db.sessions.createIndex({createdAt:1},{expireAfterSeconds:3600})	Auto-expire docs after 1 hour
Partial index	createIndex({price:1},{partialFilterExpression:{active:true}})	Index subset of docs

```
// Average rating per category – full aggregation pipeline
db.products.aggregate([
{ $match: { isActive: true } }, // stage 1: filter
{ $unwind: "$reviews" }, // stage 2: flatten array
{ $group: { // stage 3: group
_id: "$category",
avgRating: { $avg: "$reviews.rating" },
totalReviews: { $sum: 1 },
productCount: { $addToSet: "$_id" }
}},
{ $project: { // stage 4: shape output
category: "$_id",
avgRating: { $round: ["$avgRating", 1] },
totalReviews: 1,
productCount: { $size: "$productCount" }
}},
{ $sort: { avgRating: -1 } }, // stage 5: sort
{ $limit: 10 } // stage 6: top 10
]);
```

$lookup (Join) and $facet (Multi-Aggregation)

```
// $lookup: join products with categories
db.products.aggregate([
{ $lookup: {
from: "categories",
localField: "category_id",
foreignField: "_id",
as: "categoryInfo"
}},
{ $unwind: "$categoryInfo" },
{ $project: { name:1, "categoryInfo.name":1, price:1 } }
]);
// $facet: compute multiple aggregations in one pass
db.products.aggregate([
{ $match: { isActive: true } },
{ $facet: {
priceRanges: [{ $bucket: { groupBy: "$price",
boundaries: [0,1000,5000,10000,Infinity],
default: "Other" }}],
topBrands: [{ $group: { _id: "$attributes.brand", count: {$sum:1} }},
{ $sort: { count:-1} }, { $limit: 5}],
total: [{ $count: "n" }]
}}
]);
```

06-06 Cassandra – Wide-Column Model

CQL Schema for Order Events

```
-- Design principle: model for your read queries
-- Query: "get last 50 events for customer X"
CREATE TABLE shopverse.order_events (
customer_id BIGINT,
event_time TIMEUUID,
order_id BIGINT,
event_type TEXT,
payload TEXT,
PRIMARY KEY (customer_id, event_time)
) WITH CLUSTERING ORDER BY (event_time DESC)
AND compaction = {'class': 'TimeWindowCompactionStrategy',
'compaction_window_unit': 'DAYS',
'compaction_window_size': 1};
```

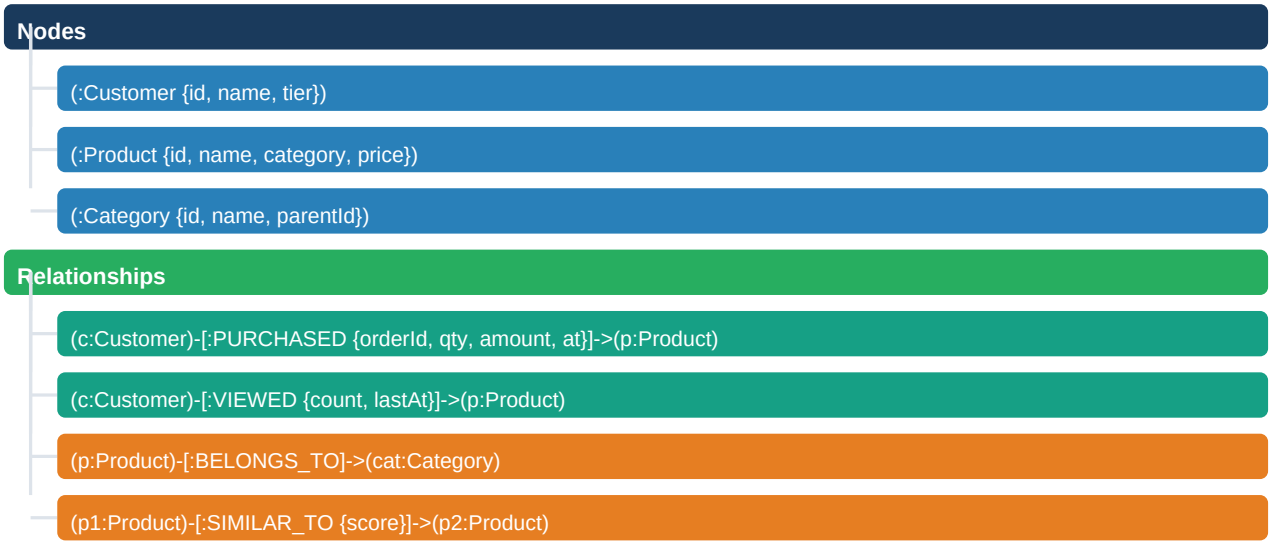
Cassandra Key Concepts

Concept	Explanation	ShopVerse
Partition Key	Which node stores the row	customer_id
Clustering Key	Row order within partition	event_time DESC
Replication Factor	Replica count across nodes	RF=3 in prod
Consistency Level	Replicas that must ACK	QUORUM write, LOCAL_ONE read
Tombstones	Deletion markers; accumulate and slow reads	Use TTL instead of DELETE
TWCS	TimeWindowCompactionStrategy	Best for time-series

Anti-Patterns in Cassandra

Anti-Pattern	Problem	Fix
Unbounded partition	Partition > 100MB causes hotspot	Add bucket (e.g. week_id) to partition key
ALLOW FILTERING	Full table scan	Redesign table for the query
Row-by-row DELETE	Tombstone accumulation	Use TTL on insert instead
Counter without idempotency	Lost updates on retry	Use regular BIGINT + conditional update

Product Recommendation Graph Model




```
// "Customers who bought this also bought" – Cypher
MATCH (target:Product {id: $productId})
MATCH (other:Customer)-[:PURCHASED]->(target)
MATCH (other)-[:PURCHASED]->(rec:Product)
WHERE rec.id <> $productId
AND NOT (:Customer {id: $userId})-[:PURCHASED]->(rec)
RETURN rec, COUNT(*) AS score
ORDER BY score DESC LIMIT 10;
```

```
// Collaborative filtering – find customers like me
MATCH (me:Customer {id: $userId})-[:PURCHASED]->(p:Product)
MATCH (similar:Customer)-[:PURCHASED]->(p)
WHERE similar.id <> $userId
WITH similar, COUNT(p) AS shared ORDER BY shared DESC LIMIT 5
MATCH (similar)-[:PURCHASED]->(rec:Product)
WHERE NOT (me)-[:PURCHASED]->(rec)
RETURN rec, COUNT(*) AS score ORDER BY score DESC LIMIT 10;
```

Index Mapping for Product Search

```
{
"mappings": {
"properties": {
"name": { "type": "text", "analyzer": "english" },
"description": { "type": "text", "analyzer": "english" },
"category": { "type": "keyword" },
"brand": { "type": "keyword" },
"price": { "type": "float" },
"tags": { "type": "keyword" },
"isActive": { "type": "boolean" },
"ratings.avg": { "type": "float" },
"suggest": { "type": "completion" } // autocomplete
}
}
}
```

Search Query with Facets

```
// Spring Data Elasticsearch – search with filters and facets
NativeSearchQuery query = new NativeSearchQueryBuilder()
.withQuery(QueryBuilders.multiMatchQuery(keyword, "name","description"))
.withFilter(QueryBuilders.termQuery("isActive", true))
.withFilter(QueryBuilders.rangeQuery("price").gte(minPrice).lte(maxPrice))
.addAggregation(AggregationBuilders.terms("brands").field("brand").size(20))
.addAggregation(AggregationBuilders.range("price_ranges").field("price")
.addRange(0, 1000).addRange(1000, 5000).addRange(5000, Double.MAX_VALUE))
.withPageable(PageRequest.of(page, size))
.build();
```

Elasticsearch vs PostgreSQL FTS

Feature	Elasticsearch	PostgreSQL FTS
Relevance scoring	TF-IDF / BM25 with boosting	ts_rank (basic)
Faceted search	Native aggregations	Manual GROUP BY
Autocomplete	completion suggester	trigram index
Multilingual	Per-field analyzers	Per-index config
Scalability	Horizontal (shards)	Vertical mainly
Consistency	Eventually consistent	Strongly consistent

06-09 **CAP Theorem & Consistency Models**

CAP Theorem

CAP: choose any 2 of 3 in distributed system

- C – Consistency: every read sees most recent write
- A – Availability: every request gets a (non-error) response
- P – Partition Tolerance: system works despite network splits

PostgreSQL → CP (sacrifices availability during partition)

Redis Cluster → AP (may serve stale data during split)

Cassandra → AP (tunable: QUORUM gives stronger consistency)

MongoDB → CP (primary election during partition)

Elasticsearch → AP (near-real-time; eventual consistency)

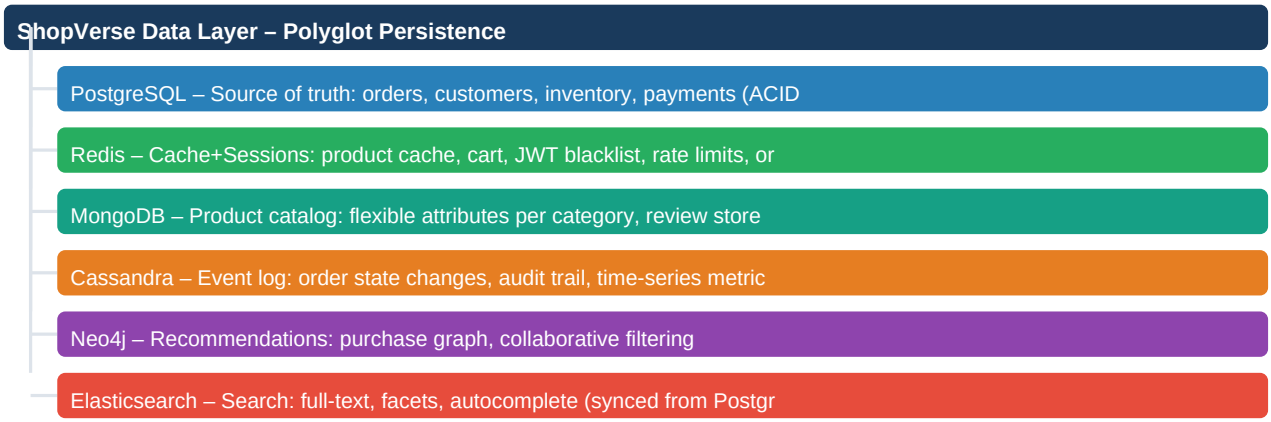
Consistency Models Reference

Model	Guarantee	ShopVerse System
Strong/Linearizable	All reads see latest write; globally ordered	PostgreSQL (within one primary)
Sequential	Operations in program order; not real-time	Kafka partition (within partition)
Causal	Causally related ops in order	MongoDB causal sessions
Read-your-writes	Writer always sees own writes	Redis single-node, Cassandra QUORUM
Eventual	Replicas converge given no new writes	Cassandra ONE, Elasticsearch

06-10 SQL vs NoSQL Decision Matrix

Requirement	Choose	Reason
ACID transactions, complex joins, referential integrity	PostgreSQL	Full SQL, FK constraints, relational model
Sub-millisecond cache, sessions, counters, pub/sub	Redis	In-memory; O(1) data structures
Flexible schema, rich document queries, aggregation pipeline	MongoDB	Schema-less; powerful aggregations
High-write time-series, append-only, multi-DC replication	Cassandra	Tunable consistency; linear write scale
Relationship traversal, social graph, recommendations	Neo4j	Cypher; native graph; fast at depth 3+
Full-text search, faceting, autocomplete, relevance scoring	Elasticsearch	Inverted index; BM25 scoring; aggregations

ShopVerse Multi-Database Rationale



06-11 Data Modelling Comparison

Database	Order Representation	Trade-offs
PostgreSQL	orders + order_items tables (normalised, FK to join)	ACID, joins, no duplication, complex queries
MongoDB	Single order document with embedded items array	Fast read (1 query), flexible schema, no joins
Cassandra	order_events partitioned by customer_id + event_time	Write-optimised, time-series, no complex queries
Redis	String order:{id}→JSON | Hash cart:{userId}	Sub-ms, volatile, cache-only, no relationships
Neo4j	(Customer)-[:PLACED]->(Order)-[:CONTAINS]->(Product)	Relationship traversal, recommendations

Embedding vs Referencing in MongoDB

Strategy	When to Use	Example
Embed	Data always accessed together; bounded size; owned	Order with items (fetch order = fetch items)
Reference (id only)	Data accessed independently; unbounded size; shared	Product referenced by many orders (shared catalog)
Hybrid	Core data embedded; extended data referenced	Order embeds summary; full product ref by ID

Redis Topology Options

Topology	Nodes	Reads from	Sharding	ShopVerse
Standalone	1 primary	Primary	None	Dev/test only
Sentinel	1 primary + N replicas + 3 sentinels	Primary or replica	None	HA without sharding
Cluster	6+ (3 primary + 3 replica)	Primary or replica per slot	16384 hash slots	Production – scale out

```
# application.yml – Redis Cluster config
spring.data.redis:
cluster:
nodes:
- redis-node-1:6379
- redis-node-2:6379
- redis-node-3:6379
max-redirects: 3
lettuce:
cluster:
refresh:
adaptive: true # auto-detect topology changes
period: 60s
```

Redis Persistence Options

Mode	Config	Durability	Performance	ShopVerse Use
No persistence	save ""	None (volatile)	Highest	Dev cache only
RDB (snapshot)	save 900 1	Last snapshot (minutes old)	High	Non-critical cache
AOF (append-only file)	appendonly yes	Per-second or per-write	Medium	Session store
RDB + AOF	Both enabled	Best (AOF with RDB for fast recovery)	Medium	Rate limits, cart data

```
# Redis maxmemory policy – what to evict when full
maxmemory 2gb
maxmemory-policy allkeys-lru # evict least-recently-used keys
# Options: noeviction, allkeys-lru, volatile-lru, allkeys-lfu,
# volatile-ttl, allkeys-random
```

■ Use allkeys-lru for a pure cache (all keys can be evicted). Use volatile-lru if you mix cache keys (with TTL) and persistent keys (without TTL).

BP Best Practices & Anti-Patterns

Do

- ✓ Set TTL on ALL Redis cache keys – use allkeys-lru eviction policy for pure cache.
- ✓ Use Lua scripts for atomic Redis operations requiring read-modify-write.
- ✓ Use Redis Streams + consumer groups for reliable event delivery (vs fire-and-forget pub/sub).
- ✓ Design Cassandra tables query-first – partition key must match your WHERE clause.
- ✓ Use TWCS for Cassandra time-series tables – compaction aligned with time windows.
- ✓ Use MongoDB $facet for catalog faceted search – single aggregation pass for multiple counts.
- ✓ Use Elasticsearch for full-text search; sync from Postgres via Debezium CDC (not dual-write).
- ✓ Use Neo4j only for relationship-heavy queries; keep business data in Postgres.
- ✓ Monitor Redis memory with INFO memory; set maxmemory to prevent OOM kill.

Anti-Patterns

- ■ Using Redis as primary DB without persistence – data is volatile; loss on restart.
- ■ No TTL on Redis keys – unbounded memory growth; eventual eviction or OOM.
- ■ MongoDB without indexes on filter fields – collection scan O(n) on every query.
- ■ Cassandra ALLOW FILTERING – full table scan; never use in production.
- ■ Unbounded Cassandra partitions (no time-bucket) – partitions > 100MB degrade reads.
- ■ Elasticsearch as source of truth – eventually consistent; use Postgres as master.
- ■ Dual-write to multiple databases without CDC – consistency gaps under failure.
- ■ Storing large binary blobs in Redis – memory waste; use S3 + Redis key for reference.

QR Quick Reference

Database	Best For	ShopVerse Primary Use	Key Config
Redis	Sub-ms cache, sessions, pub/sub, atomic ops	Product cache, cart (hash), rate limit counters	allkeys-lru; maxmemory 2gb; AOF for sessions
MongoDB	Flexible documents, aggregation	Product catalog attributes, reviews	Replica set RF=3; compound index on (category,price)
Cassandra	High-write time-series append-only	Order event log, audit trail	RF=3; TWCS; QUORUM write; TTL not DELETE
Neo4j	Graph traversal, recommendations	Purchase graph, collaborative filtering	Causal cluster; index on :Product(id); :Customer(id)
Elasticsearch	Full-text, facets, autocomplete	Product search	text+keyword mappings; completion suggester; refresh_inte

Interview Quick-Check

Question	Concise Answer
When Redis over PostgreSQL for cache?	Sub-ms latency need; data fits in memory; stale-for-TTL acceptable; can rebuild from Postgres on r
MongoDB embedding vs referencing?	Embed: always accessed together, bounded size. Reference: shared across documents, large/grow
Why TWCS for Cassandra?	Compacts SSTable files within time windows. Old windows become immutable – no rewrite cost. Id
CAP and Cassandra choices?	Cassandra is AP by default. Use QUORUM for CP-like behaviour (requires majority of replicas aliv
Elasticsearch vs Postgres FTS?	ES: BM25 relevance, faceting, autocomplete, horizontal scale. Postgres FTS: simpler, consistent, r
Redis pub/sub vs streams?	Pub/sub: fire-and-forget, no persistence. Streams: persistent, consumer groups, replay, acknowled